

Skill 12

1. **Task1:**To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

Code:

```
#include <stdio.h>
```

```
#include <signal.h>
```

```
#include <unistd.h>
```

```
void handler(int sig)
```

```
{
```

```
    printf("\nSignal %d received. Handler executed.\n", sig);
```

```
}
```

```
int main()
```

```
{
```

```
    signal(SIGUSR1, handler);
```

```
    printf("PID: %d\n", getpid());
```

```
    printf("Waiting for signal...\n");
```

```
    sleep(8);
```

```
    return 0;
```

```
}
```

Outupt:

```
ashwithreddy_earla@AshwithReddy:~$ nano Skill12.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill12.c -o Skill12
ashwithreddy_earla@AshwithReddy:~$ ./Skill12
PID: 486
Waiting for signal...
ashwithreddy_earla@AshwithReddy:~$ |
```

1.

Task2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

Code: #include <stdio.h>

#include <signal.h>

#include <unistd.h>

void handle_sigint(int sig)

{

printf("\nSIGINT received. Foreground job terminated safely.\n");

}

int main()

{

signal(SIGINT, handle_sigint);

printf("PID: %d\n", getpid());

printf("Press Ctrl+C...\n");

```
    for (int i = 1; i <= 5; i++)  
    {  
        printf("Running: %d\n", i);  
        sleep(1);  
    }  
  
    return 0;  
}
```

OUTPUT:

```
ashwithreddy_earla@AshwithReddy:~$ nano Skill12.1.c  
ashwithreddy_earla@AshwithReddy:~$ gcc Skill12.1.c -o Skill12.1  
ashwithreddy_earla@AshwithReddy:~$ ./Skill12.1  
PID: 575  
Press Ctrl+C...  
Running: 1  
Running: 2  
Running: 3  
Running: 4  
Running: 5  
ashwithreddy_earla@AshwithReddy:~$ |
```

Task3: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

Code:

```
#include <stdio.h>  
  
#include <signal.h>  
  
#include <unistd.h>
```

```
void handle_sigstsp(int sig)
{
    printf("\nSIGTSTP received. Job state preserved.\n");
}

int main()
{
    signal(SIGTSTP, handle_sigstsp);

    printf("PID: %d\n", getpid());
    printf("Press Ctrl+Z...\n");

    for (int i = 1; i <= 5; i++)
    {
        printf("Running: %d\n", i);
        sleep(1);
    }

    return 0;
}
```

OUTPUT :

```
ashwithreddy_earla@AshwithReddy:~$ nano Skill12.2.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill12.2.c -o Skill12.2
ashwithreddy_earla@AshwithReddy:~$ ./Skill12.2
PID: 665
Press Ctrl+Z...
Running: 1
Running: 2
Running: 3
Running: 4
Running: 5
ashwithreddy_earla@AshwithReddy:~$ |
```

Task4: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

Code: `#include <stdio.h>`

`#include <unistd.h>`

`#include <sys/types.h>`

`#include <sys/wait.h>`

`int main()`

`{`

`pid_t pid = fork();`

`if (pid == 0)`

`{`

`setpgid(0, 0);`

`printf("Child PID: %d\n", getpid());`

```
    printf("Process Group ID: %d\n", getpgrp());

    sleep(2);

    return 0;
}

wait(NULL);

printf("Parent PID: %d\n", getpid());
printf("Child process group completed.\n");

return 0;
}
```

OUTPUT:

```
ashwithreddy_earla@AshwithReddy:~$ nano Skill12.3.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill12.3.c -o Skill12.3
ashwithreddy_earla@AshwithReddy:~$ ./Skill12.3
Child PID: 730
Process Group ID: 730
Parent PID: 729
Child process group completed.
ashwithreddy_earla@AshwithReddy:~$ |
```